

Thread Class in C# with Examples

In this article, I am going to discuss **Thread Class in C#** with Examples. Please read our previous article before proceeding to this article where we discussed the basic concepts of [Multithreading in C#](#) with Examples. As part of this article, we are going to discuss the following pointers in detail with examples.

1. [What is Thread Class in C#?](#)
2. [Properties, Methods, and Constructors of Thread class.](#)
3. [Understanding the different Constructors of Thread Class in detail.](#)
4. [Why is the Constructor of Thread Class taking a Parameter of Delegate Type?](#)
5. [Understanding ThreadStart delegate in C#.](#)
6. [Thread Function with Parameter.](#)
7. [Understanding ParameterizedThreadStart Delegate in C#.](#)
8. [When to use ParameterizedThreadStart over ThreadStart delegate?](#)
9. [What are the Problems with the ParameterizedThreadStart delegate in C#?](#)
10. [How to Overcome the Problems of ParameterizedThreadStart delegate in C#?](#)

What is Thread Class in C#?

The Thread class in C# is responsible for creating custom threads. With the help of the Thread class, we can create both foreground and background threads. What are the [foreground and background threads](#) that we are going to discuss in our coming articles? The Thread class also allowed us to set the priority of a thread. In our coming session, we will discuss [how to set the priority of a thread](#).

If you go to the definition of the Thread class, then you will see that is a sealed class and as it is a sealed class, this class cannot be inherited i.e. further inheritance is not possible. The Thread class in C# provides lots of useful properties, methods, and constructors, which we are going to discuss in this article.

Thread Class Properties in C#:

The Thread class in C# provides lots of properties. Some of the important properties are as follows:

1. **CurrentThread**: This property is used to get the currently running thread. It returns a Thread that is the representation of the currently running thread.
2. **IsAlive**: This property is used to get a value indicating the execution status of the current thread. It returns true if this thread has been started and has not terminated normally or aborted; otherwise, false.
3. **IsBackground**: This property is used to get or set a value indicating whether or not a thread is a background thread. It returns true if this thread is or is to become a background thread; otherwise, false.
4. **ManagedThreadId**: This property is used to get a unique identifier for the current managed thread. It returns an integer that represents a unique identifier for this managed thread.
5. **Name**: This property is used to get or set the name of the thread. It returns a string containing the name of the thread, or null if no name was set.
6. **Priority**: This property is used to get or set a value indicating the scheduling priority of a thread. It returns one of the System.Threading.ThreadPriority values. The default value is System.Threading.ThreadPriority.Normal.
7. **ThreadState**: This property is used to get a value containing the states of the current thread. It returns one of the System.Threading.ThreadState values indicate the state of the current thread. The initial

value is Unstarted.

8. **IsThreadPoolThread**: This property is used to get a value indicating whether or not a thread belongs to the managed thread pool. It returns true if this thread belongs to the managed thread pool; otherwise, false.

Thread Class Methods in C#:

The Thread class in C# provides lots of methods. Some of the important methods are as follows:

1. **Abort()**: This method is used to terminate the thread. Raises a ThreadAbortException in the thread on which it is invoked, to begin the process of terminating the thread. Calling this method usually terminates the thread.
2. **Interrupt()**: This method is used to interrupt a thread that is in the WaitSleepJoin thread state.
3. **Join()**: This method is used to block the calling thread until the thread represented by this instance terminates while continuing to perform standard COM and SendMessage pumping.
4. **ResetAbort()**: This method is used to an System.Threading.Thread.Abort(System.Object) requested for the current thread.
5. **Resume()**: This method is used to resume a thread that has been suspended.
6. **Sleep(Int32)**: This method is used to suspend the current thread for the specified number of milliseconds.
7. **Start()**: This method causes the operating system to change the state of the current instance to System.Threading.ThreadState.Running.
8. **Suspend()**: This method is used to either suspend the thread or if the thread is already suspended, have no effect
9. **Yield()**: This method causes the calling thread to yield execution to another thread that is ready to run on the current processor. The operating system selects the thread to yield to. It returns true if the operating system switched execution to another thread; otherwise, false.

Thread Class Constructors in C#:

The Thread class in C# provides the following 4 constructors.

1. **Thread(ThreadStart start)**: It initializes a new instance of the Thread class. Here, the parameter start specifies a ThreadStart delegate that represents the methods to be invoked when this thread begins executing. It will throw ArgumentNullException, the start parameter is null.
2. **Thread(ParameterizedThreadStart start)**: It initializes a new instance of the Thread class, specifying a delegate that allows an object to be passed to the thread when the thread is started. Here, the parameter start specifies a delegate that represents the methods to be invoked when this thread begins executing. It will throw ArgumentNullException, the start parameter is null.
3. **Thread(ThreadStart start, int maxStackSize)**: It initializes a new instance of the Thread class, specifying the maximum stack size for the thread. Here, the parameter start specifies a ThreadStart delegate that represents the methods to be invoked when this thread begins executing. And the parameter maxStackSize specifies the maximum stack size, in bytes, to be used by the thread, or 0 to use the default maximum stack size specified in the header for the executable. Important For partially trusted code, maxStackSize is ignored if it is greater than the default stack size. No exception is thrown.
4. **Thread(ParameterizedThreadStart start, int maxStackSize)**: It initializes a new instance of the Thread class, specifying a delegate that allows an object to be passed to the thread when the thread is started and specifies the maximum stack size for the thread.

Note: In this article, I am going to discuss the Thread class constructors in detail with examples and the methods and properties of thread class are going to be discussed in our subsequent articles.

Example to Understand Constructors of Thread Class in C#.

Let us understand the constructors of the thread class in C# with an example. Please have a look at the following code. It's a very simple program and here we are executing the DisplayNumbers function using a different thread.

```
using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Thread t1 = new Thread(DisplayNumbers);
            t1.Start();
            Console.Read();
        }
        static void DisplayNumbers()
        {
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine("Method1 :" + i);
            }
        }
    }
}
```

As you can see in the above code, here, we created an instance of the Thread class and to the constructor of the Thread class, we passed the method name that we want the thread to execute. The following line of code exactly does the same.

Thread t1 = new Thread(DisplayNumbers);

Constructors of Thread Class in C#:

As discussed earlier, in C#, the Thread class contains four constructors. If you go to the definition of Thread class then you will see the Constructors as shown below.

```
public sealed class Thread : CriticalFinalizerObject, _Thread
{
    ...public Thread(ThreadStart start);
    ...public Thread(ParameterizedThreadStart start);
    ...public Thread(ThreadStart start, int maxStackSize);
    ...public Thread(ParameterizedThreadStart start, int maxStackSize);
}
```

Now you may have one question, the Thread class constructor which takes one parameter is either of the type ThreadStart or ParameterizedThreadStart, but in our example, we are passing the method name as a parameter to the Thread class Constructor and it works, how? To understand this, let us go to the definition of ThreadStart and you will see that ThreadStart is actually a delegate as shown in the below image.

```
namespace System.Threading
{
    ...public delegate void ThreadStart();
}
```

Why is the Constructor of the Thread class Taking a Parameter of Delegate Type?

As we already discussed, the main objective of creating a Thread in C# is to execute a function. A delegate is a type-safe function pointer. It means the delegate points to a function and when we invoke the delegate, the function which is pointed by the delegate is going to be executed.

In simple words, we can say that all the threads that we create require an entry point (i.e. a pointer to the function) from where it should execute. This is the reason why threads always require a delegate. If you want to learn [Delegates in C#](#) with Examples, then I strongly recommended you read our [Delegates in C#](#) article.

Note: We already discussed in our Delegates article that the signature of the delegate should and must be the same as the signature of the method it points to.

As you can see, the ThreadStart delegate does not take any parameter and the return type is void. In our example, the DisplayNumbers() function signature is the same as the ThreadStart delegate signature as the DisplayNumbers() function return type is void as well as it does not take any parameters.

Thread t1 = new Thread(DisplayNumbers);

So, the above thread instance creation statement is implicitly converted to the ThreadStart delegate instance. So, you can write the above statement as shown in the below image and it will work.

```
Thread t1 = new Thread(DisplayNumbers);
```



```
ThreadStart obj = new ThreadStart(DisplayNumbers);  
Thread t1 = new Thread(obj);
```

As you can see in the above image, it is a two steps process. First, we need to create the ThreadStart Delegate Instance and while creating the instance, to its constructor we need to pass the method name that we want to execute. In the second step, to the Constructor of Thread class, we need to pass the ThreadStart instance as a parameter. And when we start the Thread execution by calling the Start method on the thread instance, internally it will invoke the ThreadStart delegate instance which internally start the execution of the DisplayNumbers method.

Example to Understand ThreadStart Delegate in C#:

In the below example, first, we are creating an instance of ThreadStart delegate and to the constructor of ThreadStart delegate, we pass the DisplayNumbers function as a parameter. Then we create an instance of the Thread class and to the constructor of the Thread class, we pass the ThreadStart delegate instance as a parameter that points to the DisplayNumbers function. Finally, when we call the Start method on the Thread instance, which will invoke the ThreadStart delegate instance and the delegate instance will invoke the method it points to i.e. the DisplayNumbers function is going to start its execution.

```
using System.Threading;  
using System;  
namespace ThreadingDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Creating the ThreadStart Delegate instance by passing the  
            //method name as a parameter to its constructor  
            ThreadStart obj = new ThreadStart(DisplayNumbers);  
            //Passing the ThreadStart Delegate instance as a parameter to its  
            Thread t1 = new Thread(obj);  
            t1.Start();  
            Console.Read();  
        }  
        static void DisplayNumbers()  
        {  
            for (int i = 1; i <= 5; i++)  
            {  
                Console.WriteLine("Method1 :" + i);  
            }  
        }  
    }  
}
```

```
}
```

You can also combine the above two statements into a single statement as shown below.

```
ThreadStart obj = new ThreadStart(DisplayNumbers);  
Thread t1 = new Thread(obj);  
↓  
Thread t1 = new Thread(new ThreadStart(DisplayNumbers));
```

Creating Thread Class Instance using Anonymous Method in C#:

It is also possible to create a Thread class instance by using the anonymous method as shown in the below example. We know that Anonymous methods are created by using the delegate keyword and they are assigned to a type of delegate.

```
using System.Threading;  
using System;  
namespace ThreadingDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            //Creating Thread Class Instance using Lambda Expression  
            Thread t1 = new Thread(delegate ()  
            {  
                for (int i = 1; i <= 5; i++)  
                {  
                    Console.WriteLine("Method1 :" + i);  
                }  
            });  
            t1.Start();  
            Console.Read();  
        }  
    }  
}
```

Creating Thread Class Instance using Lambda Expression in C#:

It is also possible to create the thread class instance by using the lambda expression as shown in the below example. We know that Lambda Expressions are created by using the => Lambda Operator.

```
using System.Threading;  
using System;  
namespace ThreadingDemo
```

```

{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating Thread Class Instance using Lambda Expression
            Thread t1 = new Thread(() =>
            {
                for (int i = 1; i <= 5; i++)
                {
                    Console.WriteLine("Method1 :" + i);
                }
            });
            t1.Start();
            Console.Read();
        }
    }
}

```

Thread Function with Parameter in C#:

The examples that we have worked on so far do not take any parameters. That is the method that we want to execute by the custom thread that does not take any parameter. Now, let us proceed and try to understand how to work with the thread function which takes the input parameter.

Let's change the DisplayNumbers() method implementation to take one parameter. Now, this method takes one input parameter of the object type. And then convert that object type to an integer value and then print the numbers up to that value starting from 1. For a better understanding, please have a look at the below example.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //DisplayNumbers is now a non-static method, so we need to
            //refer it by using the instance
            Program obj = new Program();
            Thread t1 = new Thread(obj.DisplayNumbers);
            t1.Start(5);
            Console.Read();
        }
        public void DisplayNumbers(object Max)
        {
            int Number = Convert.ToInt32(Max);
            for (int i = 1; i <= Number; i++)
            {

```

```

        Console.WriteLine("Method1 :" + i);
    }
}
}

```

When the thread method or thread function (the function which is going to be executed by the custom thread) takes one parameter, then the Thread class uses the ParameterizedThreadStart delegate internally. The definition of the ParameterizedThreadStart delegate in C# is shown in the below image.

```

namespace System.Threading
{
    ...public delegate void ParameterizedThreadStart(object obj);
}

```

As you can see the ParameterizedThreadStart delegate is taking one parameter of object type and like the ThreadStart delegate it also does not return any value. Now the DisplayNumbers() method signature is the same as the signature of this ParameterizedThreadStart delegate. So, the .NET Framework internally converts the statements as shown in the below image.

```

Thread t1 = new Thread(obj.DisplayNumbers);
↓
Thread t1 = new Thread(new ParameterizedThreadStart(obj.DisplayNumbers));

```

Creating the ParameterizedThreadStart Instance Manually in C#:

Let us see how to create the ParameterizedThreadStart delegate instance manually and pass that instance to the Thread class Constructor. For a better understanding, please have a look at the below example. In the below example, first, we are creating an instance of ParameterizedThreadStart delegate and to the constructor of ParameterizedThreadStart delegate, we pass the DisplayNumbers function as a parameter. Then we create an instance of the Thread class and to the constructor of the Thread class, we pass the ParameterizedThreadStart delegate instance as a parameter that points to the DisplayNumbers function. Finally, when we call the Start method by passing the value 5 on the Thread instance, it will invoke the delegate instance by passing the same value 5 and that delegate instance when invoked, will execute the DisplayNumbers function by passing the value 5.

```

using System.Threading;
using System;
namespace ThreadingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Program obj = new Program();

```



```

        ParameterizedThreadStart PTSD = new
ParameterizedThreadStart(obj.DisplayNumbers);
        Thread t1 = new Thread(PTSD);
        t1.Start(5);
        Console.Read();
    }
    public void DisplayNumbers(object Max)
    {
        int Number = Convert.ToInt32(Max);
        for (int i = 1; i <= Number; i++)
        {
            Console.WriteLine("Method1 :" + i);
        }
    }
}

```

Now run the above code and you will get the output as expected as shown in the below image.

```

Method1 :1
Method1 :2
Method1 :3
Method1 :4
Method1 :5

```

When to use ParameterizedThreadStart Delegate over ThreadStart Delegate in C#?

You need to use the ParameterizedThreadStart delegate if your method takes any values else you just need to use the ThreadStart delegate which does not take any parameter.

What are the Problems with the ParameterizedThreadStart delegate in C#?

As you can see, the parameter type of the ParameterizedThreadStart delegate is an object type. So, the parameter of the thread function is also going to be the object data type. And you cannot change the data type from object to any other type and if you try then it will give you a compile-time error. As the thread function operates on object data type, so we can pass any type of value and it accepts. As a result, the function is not going to be type-safe as we can pass any type of value. Again, as it operates on object data type, so boxing and unboxing will come into the picture and we already discussed in our [Boxing and Unboxing](#) article how it degrades our application performance. So, we should avoid object types.

Example to Pass String Value to a Thread Function in C#:

Let us try to pass a string value to the thread function and see what happens as shown below in the below example.

```

using System.Threading;
using System;
namespace ThreadingDemo

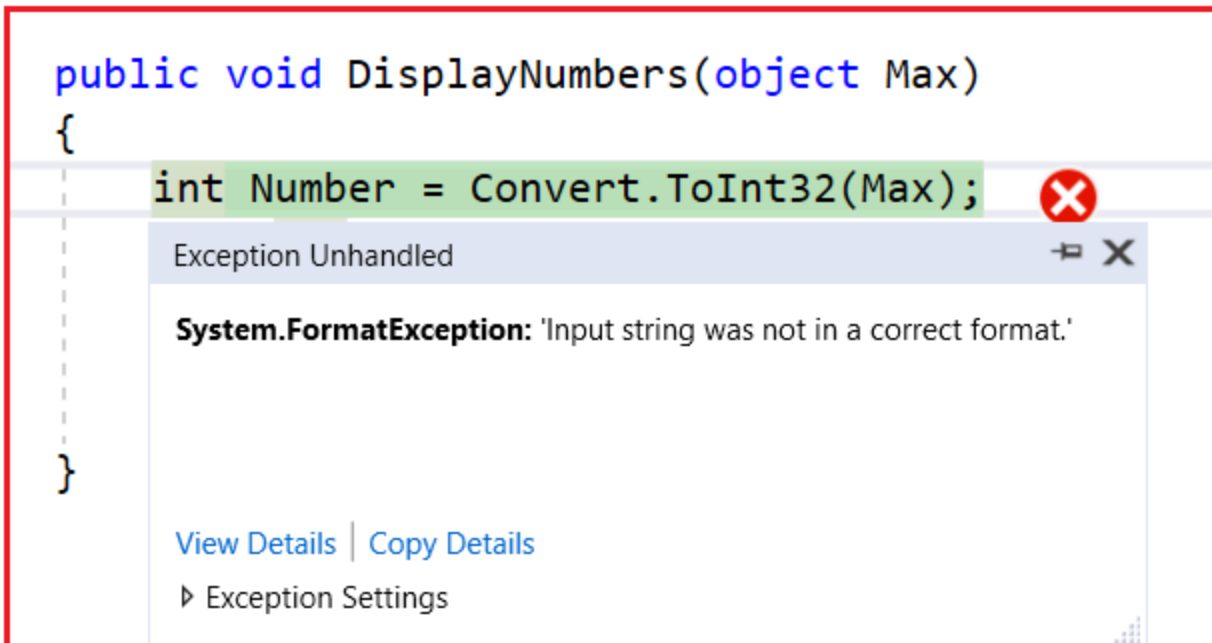
```

```

{
    class Program
    {
        static void Main(string[] args)
        {
            Program obj = new Program();
            ParameterizedThreadStart PTSD = new
ParameterizedThreadStart(obj.DisplayNumbers);
            Thread t1 = new Thread(PTSD);
            t1.Start("Hi");
            Console.Read();
        }
        public void DisplayNumbers(object Max)
        {
            int Number = Convert.ToInt32(Max);
            for (int i = 1; i <= Number; i++)
            {
                Console.WriteLine("Method1 :" + i);
            }
        }
    }
}

```

Now you will not get any compile-time error, but once you run the application, then you will get a runtime error as shown below in the below image.



In the next article, I am going to discuss [How to Pass Data to a Thread Function in a Type-Safe Manner in C#](#) with Examples. Here, in this article, I try to explain the **Thread Class in C#** with Examples. I hope you understood how to use the ThreadStart and ParameterizedThreadStart Delegates in C# Multithreading with Examples.